PRACTICAL 7-B

Object Oriented Programming

AIM:

To define custom annotations and use reflection to validate object fields at run time.

CODE SCREENSHOT:



```java
import java.lang.annotation.*;
import java.lang.reflect.*;
import java.util.*;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
@interface Id {
}

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
@interface Positive {
    String message() default "must be > 0";
}

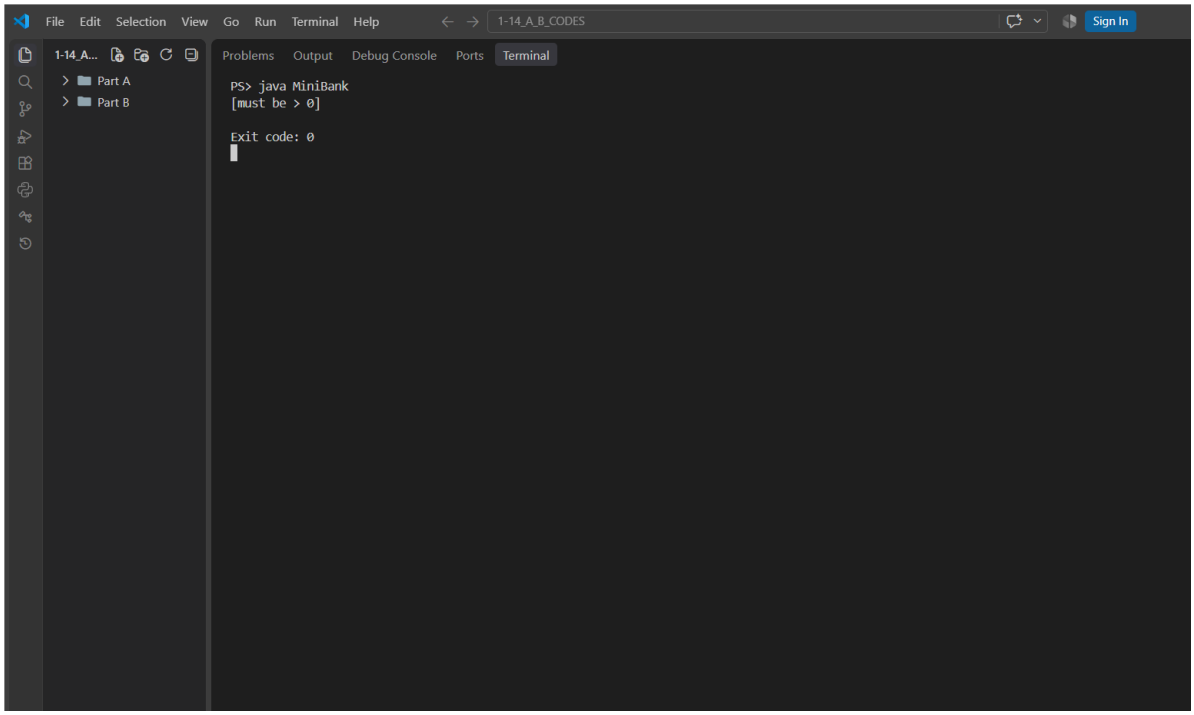
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
@interface MaxLength {
    int value();
}

class Account {
    @Id
    private String accountNumber;
    @Positive
    private long balance;
    @MaxLength(20)
    private String owner;

    Account(String n, long b, String o) {
        accountNumber = n;
        balance = b;
        owner = o;
    }
}

class AnnotationValidator {
    static String[] validate(Object obj) {
        List<String> errors = new ArrayList<>();
        try {
            for (Field f : obj.getClass().getDeclaredFields()) {
                f.setAccessible(true);
                Object v = f.get(obj);
                Positive p = f.getAnnotation(Positive.class);
                if (p != null && v instanceof Number n && n.longValue() <= 0)
                    errors.add(p.message());
                MaxLength m = f.getAnnotation(MaxLength.class);
                if (m != null && v instanceof String s && s.length() > m.value())
```

OUTPUT SCREENSHOT:



LEARNING OUTCOMES:

The student can define custom annotations and process them at run time with reflection.